

Restaurante Gourmet — Documentación del Sistema

Restaurante Gourmet es una solución web para la gestión de un restaurante: carta de platos (productos), categorías del menú, pedidos con detalle de líneas, clientes, pagos y navegación lateral dinámica según rol. El backend expone una API REST con JWT; el frontend es una SPA en Angular 19.

Arquitectura

El sistema sigue una **arquitectura en capas (N-Capas)**:

Capa	Implementación
	Presentación Angular 19 (SPA), puerto 4200
Control	Controladores REST (@RestController)
Negocio	Servicios (AuthService, PedidoService, ProductoService, etc.)
Persistencia	Spring Data JPA + Hibernate
Seguridad	JWT stateless, RBAC (ADMIN, CLIENTE)

Tecnologías detectadas

Área	Tecnología
Backend	Java 17, Spring Boot 4.0.3, Spring Security, Spring Data JPA, Hibernate 7
Frontend	Angular 19, TypeScript 5.7, RxJS, SCSS, signals
Base de datos	PostgreSQL 18 (Render — producción), H2 (tests)
Autenticación	JWT (Bearer), BCrypt
Infraestructura	Maven, mvnw, scripts PowerShell de arranque
Despliegue producción	Vercel (frontend Angular), Render (backend Spring Boot + PostgreSQL)

Puertos desarrollo: backend 8080, frontend 4200.
Producción: HTTPS en Vercel (frontend) y Render (API + base de datos).

Estructura general del proyecto

```
sistema-java-pedidos-main/  
└─ src/main/java/co/ucc/pedidos/  
|   └─ controller/    # REST API  
|   └─ service/       # Lógica de negocio  
|   └─ model/         # Entidades JPA  
|   └─ repository/    # Spring Data  
|   └─ config/        # Seguridad, CORS, inicializadores  
|   └─ security/      # JWT, filtros, UserDetails  
|   └─ dto/           # DTOs (menú, auth)  
|   └─ exception/     # Excepciones y @ControllerAdvice  
└─ pedidos-frontend/  # Cliente Angular  
    └─ src/app/  
        └─ core/      # guards, services, models  
        └─ layout/    # sidebar, main-layout  
        └─ pages/     # auth, productos, pedidos, clientes  
└─ docs/             # Esta documentación  
└─ migracion_restaurante.sql  
└─ start-backend.ps1  
└─ pedidos-frontend/start-frontend.ps1
```

Requerimientos funcionales

ID	Nombre	Descripción	Prioridad	Módulo
RQF001	Registro de cliente	Permite registrarse con email, contraseña, nombre, género y dirección; crea usuario CLIENTE y registro en cliente.	Alta	Auth
RQF002	Inicio de sesión	Login con email/contraseña; devuelve JWT con rol y idCliente.	Crítica	Auth
RQF003	Perfil de sesión	Endpoint /api/auth/me para usuario autenticado.	Media	Auth
RQF004	Carta de platos	Listado público de productos (platos) con precio, reseña, categoría e imagen.	Alta	Productos
RQF005	Gestión de platos	Alta, edición y eliminación de platos (solo ADMIN).	Alta	Productos
RQF006	Gestión de categorías	Listar (público) y crear categorías del menú (ADMIN).	Alta	Categorías
RQF007	Creación de pedidos	Cliente o admin crean pedido con detalles (producto, cantidad, precio unitario), lugar de entrega y observaciones.	Crítica	Pedidos
RQF008	Consulta de pedidos	Admin ve todos; cliente ve solo /api/pedidos/mios.	Alta	Pedidos
RQF009	Eliminación de pedidos	Borrado por ID (según permisos de rol).	Media	Pedidos
RQF010	Estados de pedido	Estados: PENDIENTE, PREPARANDO, LISTO, ENTREGADO, CANCELADO (EstadoModel).	Media	Pedidos
RQF011	CRUD de clientes	Registro, listado, búsqueda, actualización y eliminación (ADMIN).	Alta	Clientes

RQF012	Registro de pagos	Alta y consulta de pagos asociados a cliente (ADMIN).	Media	Pagos
RQF013	Menú dinámico lateral	Opciones jerárquicas desde BD; árbol en API y construcción en Angular con filtro por rol.	Media	UI / Opciones
RQF014	Dashboard inicio	Resumen con contadores de clientes (admin), productos y pedidos.	Baja	Inicio
RQF015	Reportes de pedidos	Estadísticas locales: total pedidos, ventas, promedio, por categoría (ADMIN).	Media	Pedidos

Requerimientos no funcionales

ID	Nombre	Descripción	Prioridad
RQNF001	Seguridad RBAC	Acceso HTTP según ROLE_ADMIN o ROLE_CLIENTE (Spring Security).	Crítica
RQNF002	Autenticación JWT	Sesión stateless; token 24 h; header Authorization: Bearer.	Crítica
RQNF003	Persistencia relacional	PostgreSQL con integridad referencial (pedido, detalle, producto, categoría).	Alta
RQNF004	CORS	Orígenes permitidos: localhost:4200 y 127.0.0.1:4200.	Alta
RQNF005	Validación de negocio	Precio de producto > 0; montos de pago validados; duplicados rechazados (409).	Alta
RQNF006	Migración de esquema	DatabaseSchemaFixer y ddl-auto=update para evolución de tablas restaurante.	Media
RQNF007	API REST JSON	Comunicación frontend-backend vía HTTP/JSON.	Alta
RQNF008	SPA reactiva	Angular standalone, interceptores HTTP, guards de ruta.	Media

RQNF009	Manejo centralizado de errores	GlobalExceptionHandler (400, 401, 403, 404, 409, 500).	Media
RQNF010	Usabilidad	Tema claro/oscuro (ThemeService); menú colapsable.	Baja

Historias de usuario

US001 — Registro como cliente

- **Como** visitante del restaurante, **quiero** registrarme con mi correo y contraseña **para** poder hacer pedidos en línea.
- **Criterios de aceptación:** El email no debe existir; no se permite registrar el email reservado de admin; tras registro se inicia sesión automáticamente.
- **Prioridad:** Alta.

US002 — Inicio de sesión

- **Como** cliente o administrador, **quiero** iniciar sesión **para** acceder a las funciones según mi rol.
- **Criterios de aceptación:** Credenciales válidas devuelven JWT; el frontend valida que el rol coincida con el tipo elegido (Cliente/Admin).
- **Prioridad:** Crítica.

US003 — Ver carta del restaurante

- **Como** usuario autenticado, **quiero** ver la carta con platos y precios **para** elegir qué ordenar.
- **Criterios de aceptación:** GET /productos accesible sin rol admin; se muestran categorías e imágenes placeholder.
- **Prioridad:** Alta.

US004 — Agregar plato al menú (Admin)

- **Como** administrador, **quiero** registrar un nuevo plato con nombre, precio, descripción y categoría **para** actualizar la carta.
- **Criterios de aceptación:** POST /productos con JWT admin; precio > 0; ID de producto único.
- **Prioridad:** Alta.

US005 — Gestionar categorías (Admin)

- **Como** administrador, **quiero** crear categorías (Entradas, Platos Fuertes, etc.) **para** organizar el menú.
- **Criterios de aceptación:** Listado visible; creación solo con rol ADMIN.
- **Prioridad:** Media.

US006 — Crear pedido

- **Como** cliente, **quiero** crear un pedido indicando productos, cantidades y lugar de entrega **para** recibir mi comida.
- **Criterios de aceptación:** El sistema calcula el total; vincula cliente y productos; estado inicial PENDIENTE.
- **Prioridad:** Crítica.

US007 — Ver mis pedidos

- **Como** cliente, **quiero** ver solo mis pedidos **para** hacer seguimiento.
- **Criterios de aceptación:** GET /api/pedidos/mios filtra por idCliente del token.
- **Prioridad:** Alta.

US008 — Administrar todos los pedidos

- **Como** administrador, **quiero** listar y eliminar cualquier pedido **para** controlar la operación del restaurante.
- **Criterios de aceptación:** GET /api/pedidos y DELETE solo con ADMIN.
- **Prioridad:** Alta.

US009 — Gestionar clientes (Admin)

- **Como** administrador, **quiero** crear, editar, buscar y eliminar clientes **para** mantener la base de datos de comensales.
- **Criterios de aceptación:** Rutas /clientes/** protegidas; formularios en Angular con validación.
- **Prioridad:** Media.

US010 — Reportes de ventas (Admin)

- **Como** administrador, **quiero** ver estadísticas de pedidos y ventas **para** tomar decisiones.

- **Criterios de aceptación:** Pantalla /pedidos/reportes con totales y desglose por categoría.
- **Prioridad:** Media.

US011 — Navegación por menú dinámico

- **Como** usuario, **quiero** un menú lateral según mi rol **para** acceder solo a las opciones permitidas.
- **Criterios de aceptación:** Datos desde /api/opciones; filtro en cliente oculta rutas admin para CLIENTE.
- **Prioridad:** Media.

US012 — Cerrar sesión

- **Como** usuario autenticado, **quiero** cerrar sesión **para** proteger mi cuenta en equipos compartidos.
- **Criterios de aceptación:** Se borra localStorage y redirige a /login.
- **Prioridad:** Baja.

Diagrama de Contexto - Restaurante Gourmet

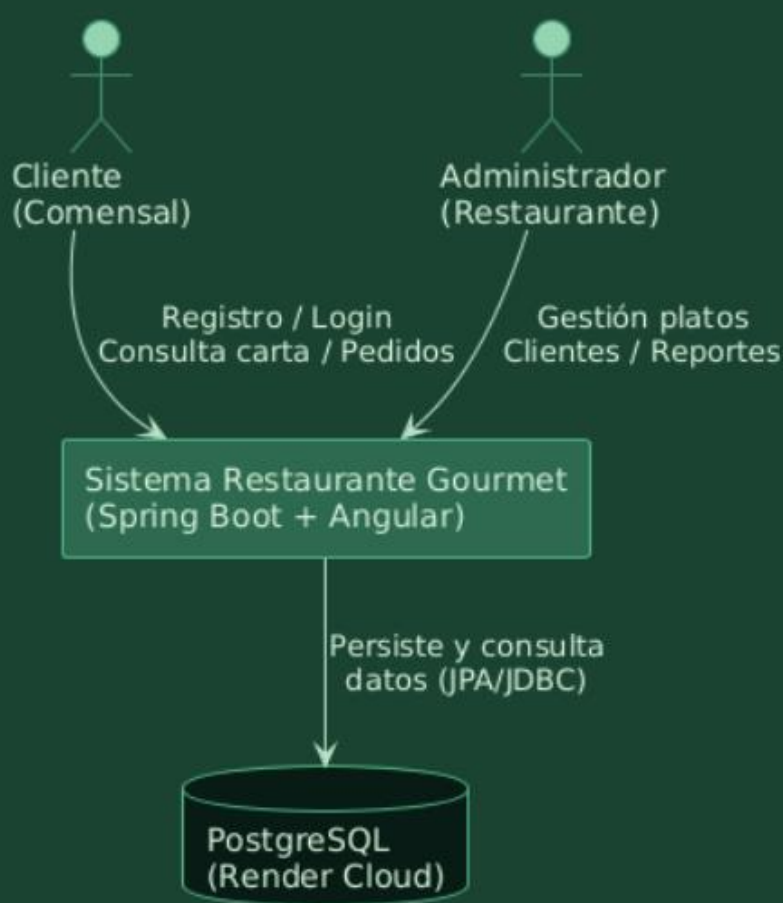
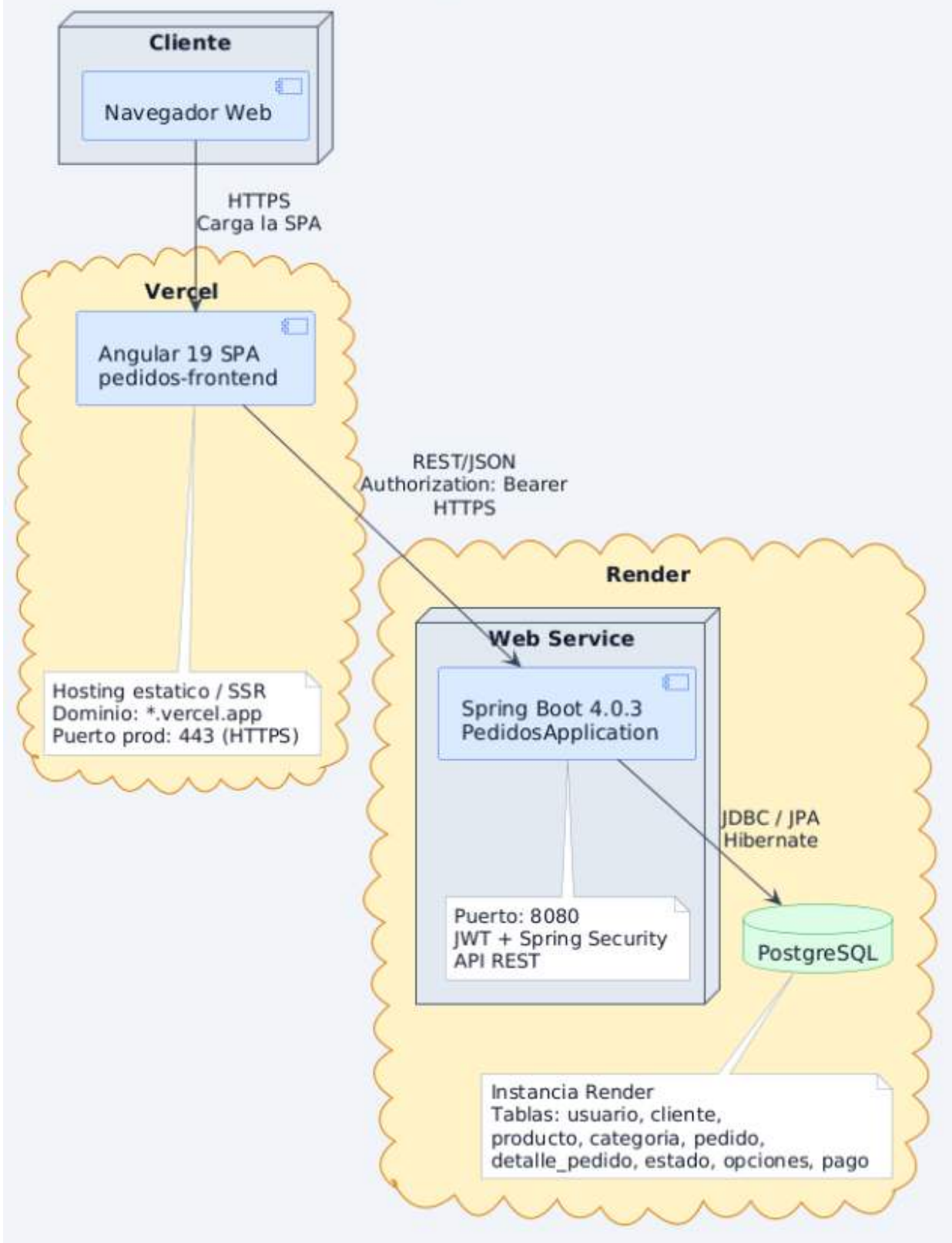


Diagrama de Despliegue - Restaurante Gourmet



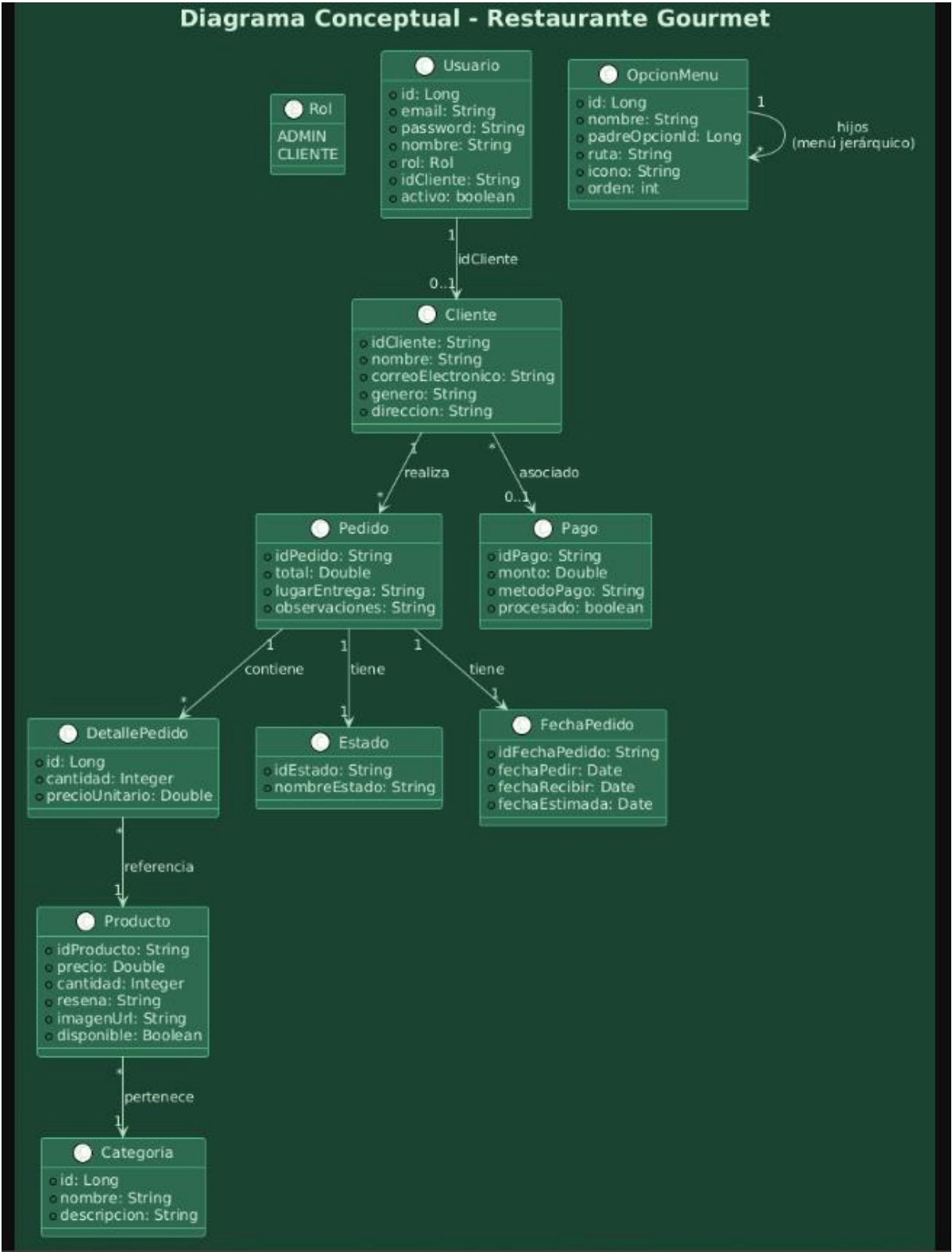
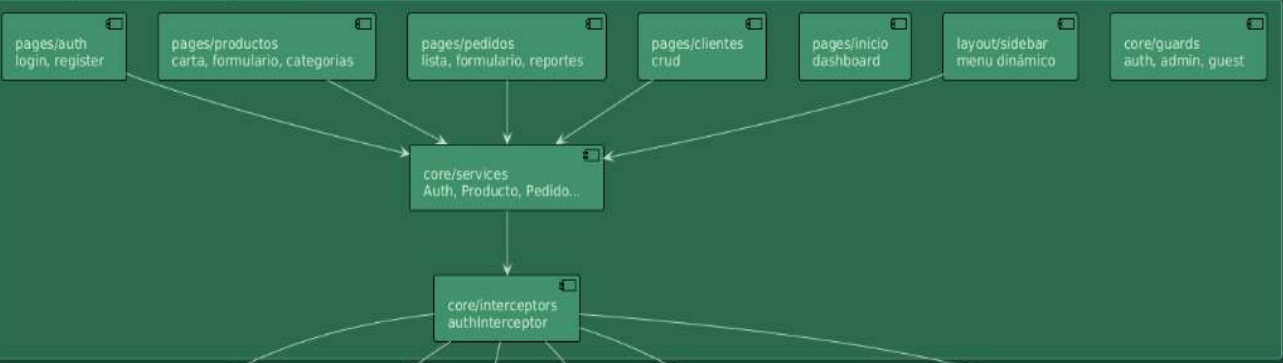


Diagrama de Desarrollo - Restaurante Gourmet

Frontend - pedidos-frontend (Angular 19)



Backend - co.ucc.pedidos

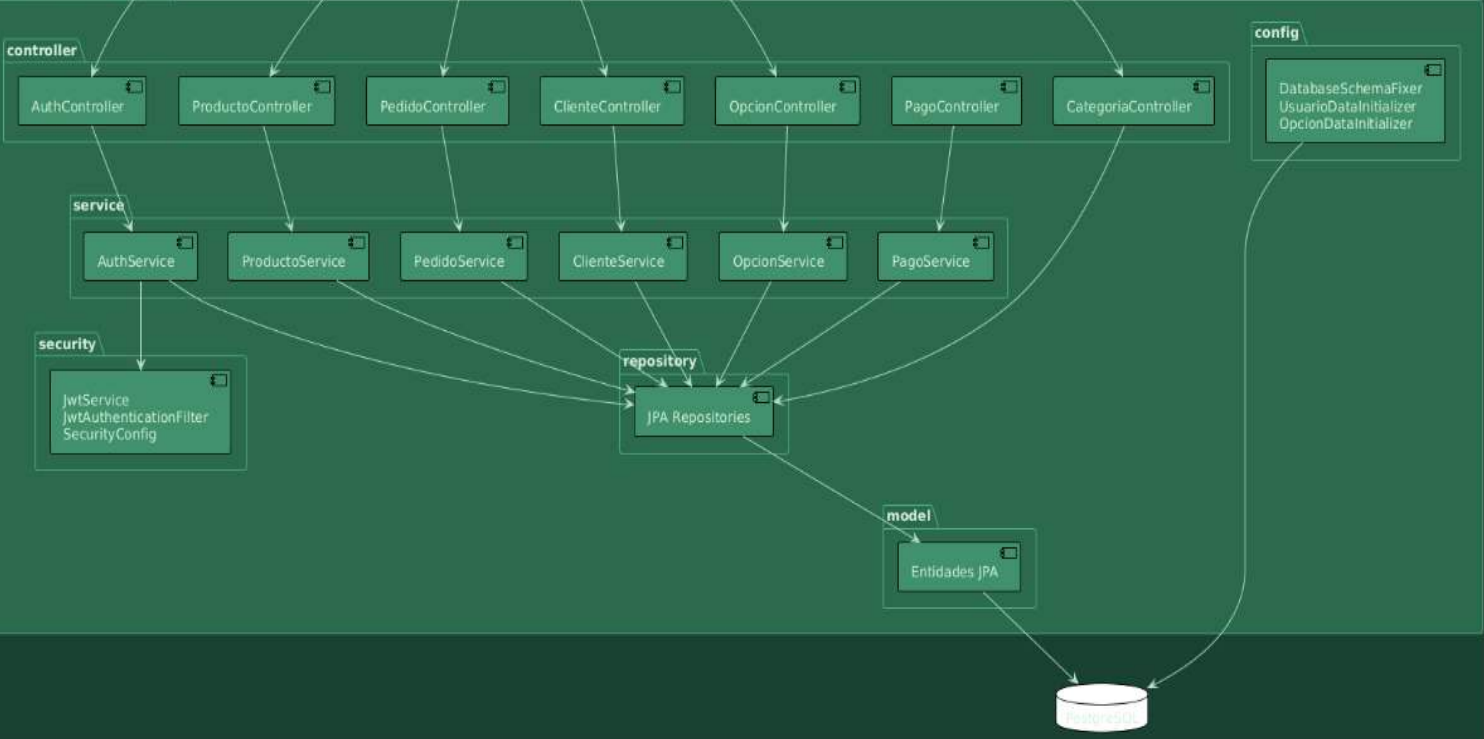


Diagrama Funcional (Casos de Uso) - Restaurante Gourmet

